Stage 8 – preparing the level editor (analysis)

As of 23 October 2026. Analysis of the decompilation (https://github.com/gcsmith/Pilotwings64Decomp, MIT license; names of functions and variables are taken from it) and measurements in mupen64plus with a hack built by Hawk's Hangar (game module, all fields open). Nothing of this is built yet.

1. Object memory of a mission (mem.c)

- taskLoadCommObj (task.c) copies every object chunk of the mission (TPAD, LPAD, LSTP, LWIN, RNGS, THER, BALS, TARG, HPAD, BTGT, PHTS, FALC, CNTG, SDFM, HOPD, OBSV) into **sLevelObjHeap, 5000 bytes at 0x8036AA60** (bump allocator mem_get 0x80312740, mem_init 0x803127BC, mem_remaining 0x803127F0; pointer sLevelObjHeapPtr 0x8034F840). Each block is `count × size`, rounded up to 8.
- **Overflow:** mem_get returns NULL and taskLoadCommObj copies the chunk to address 0 anyway (exception vectors): the game crashes. The editor has to check every mission (sum of the rounded blocks < 5000).
- Code references: 3 functions (6 hi/lo pairs) and the initial value of the pointer. Moving and enlarging the pool is cheap.
- Sizes: TPAD 0x30, LPAD 0x30, LSTP 0x24, LWIN 0x54, RNGS 0x84, THER 0x28, BALS 0x68, TARG 0x20, HPAD 0x40, BTGT 0x1C, PHTS 0x14, FALC 0xAC, CNTG 0x1C, SDFM 0x4C, HOPD 0x20, OBSV 0x10.
- Measured (pool in use after loading): GC Beginner 488, GC A1 2072, GC B1 2992, GC P1 3256 bytes. 30 rings and 20 balloons alone need 6040 bytes, so the pool is the first limit a level editor reaches.

2. Arrays per object type (counts in the COMM block are u8)

Type	Array	Max	Element	RAM	Code refs (hi/lo)	Too many
Rings	gRings	30	0x1CC	0x8036DA78	21	type off (count 0)
Balloons	gBalls	20 (incl. split balloons)	0x1AC	0x80357218	21	type off
Hover pads	gHoverPads	20	0x70	0x80369588	12	type off
Missile targets	sMissileTargets	40	0x4C	0x80378CF8	9	type off
Thermals	gThermals	16	0x18	0x8037AB58	7	type off
Landing pads (mission)	gLandingPads	14	0x18	0x8036C2E8	10	type off
Landing strips	gLandingStrips	2	0x3C	0x8036C438	6	type off
Takeoff pads	sTakeoffPads	3	0xC	0x8036C2C0	2	type off
Local winds	sLocalWinds	6	0xA8	0x8037F518	4	type off
Ball targets	sBallTgts	5	–	static	–	type off
Hopper goals	D_80368B80	5	–	0x80368B80	–	type off
Meca Hawk domains	sFalcoDomains	12 (one Meca Hawk at a time, sFalcos[1])	–	static	–	type off
Cannonball targets	sCannonTargets	1	0x48	0x8036C4B0	–	type off
Map objects (View map)	sMapObjects	60	u16	0x8036A8E8	3	message only

Type	Array	Max	Element	RAM	Code refs (hi/lo)	Too many
Radar goals	sHudState.radar.goals	10	0x10	in sHudState	–	**no check**
Radar waypoints	sHudState.radar.waypoints	70	–	in sHudState	–	not shown
Display objects	D_80263780 (kernel dobj)	100	0x40	0x80263780	34 (+26 matrix pool, +29 others)	object not drawn

- "Type off": the loader prints "too many … defined in level" and loads none of that type (safe).
- **Radar goals (original bug):** hud_8031A378 writes hopper goals, ball targets, used landing pads and landing strips into goals[10] without a check; more than 10 overwrite the waypoints behind them. The editor has to keep HOPD + BTGT + used LPAD + LSTP ⩽ 10 (or the module fixes it).
- **Display objects:** every ring, balloon, pad, target … is one dobj (uvDobjAllocIdx returns 0xFFFF when all 100 are used; the kernel functions then refuse the object). Measured in flight: GC Beginner 13, A1 39, B1 52, P1 66 (24 rings). Bigger object arrays need a bigger dobj pool (kernel: D_80263780[100], D_80265080[300] matrices, D_80269B80[300], D_80269CB0[100], D_80269D78[100], about 90 code references and 15 constants).

3. Heap and free memory (memory.c)

- The heap is a bump allocator (_uvMemAlloc, top pointer D_802B892C) from 0x8004181C up to osMemSize, jumping over the reserved blocks (uvMemInitBlocks: frame buffers 0x800DA800–0x80125800, the program block, now 0x801F1000–0x803905E0, 0x803DA800–0x80400000). uvLevelInit resets it at every level start; the island, vehicle, pilot, HUD and objects are loaded into it (uvLevelAppend of several UVLV levels).
- Free with the Expansion Pak: **5818 KB** (without it about 1.75 MB).
- Measured heap use in flight (Gyrocopter): Holiday Island 902 KB, Crescent Island 1184 KB, Little States 1729 KB (top 0x80401840: already in the Expansion Pak's memory), Ever-Frost Island 1122 KB.
- In these measurements the heap never went above 0x80402000, so the memory above about 0x80500000 is free: room for a **high data area** (e.g. 0x80780000–0x80800000) as a 7th reserved block (D_802B8830 has room for 10 blocks; module_init can add it before the first level).

4. Display list and other per-frame limits (graphics.c)

- gGfxDisplayListBase[2][4200] (0x80299280, 8 bytes per command). uvGfxEnd checks the count only at the end of a frame: above 4200 it prints "display list overflow" and returns without starting the task (the game freezes), and the second buffer has already written over gGfxViewports and the matrix stack behind it.
- Code references: 3 base loads, the stride 4200 × 8 as multiplication chains in 4 places, the constant 4200.
- Measured in flight: 1100–2500 commands (Holiday Island with rain 2319, Holiday sunset 2493, Crescent 1849).
- The developers' warning thresholds (sDisplayListNumElements and the others, 0x80249234…48, start values): display list 4000, double-buffered matrices 294 (limit 350, gGfxMstack; overflow is refused, the object uses a wrong matrix), part transforms 200, dobjs 100, effects 120, geometry vertices 800. When one is exceeded, uvGfxEnd prints "GFXEND : … <==== WINNER" through _uvDebugPrintf, so the Features option "developers' messages" shows it.
- 4200 commands cannot be predicted from the mission data (they depend on the view); measuring in an emulator is the way.

5. File tables of the kernel

- gUVBlockOffsets (0x802B53F0): UVMD[400], UVCT[128], UVTX[500] … filled at boot without a bound check (more files overwrite the next table). gLevelData (0x802C1C98, through the pointer gGfxUnkPtrs): models[400], contours[128], textures[500] …; also D_802B6E30[500] (UVTI) and D_802C60E8[500] (sprites). Retail uses 363 / 101 / 463.
- Enlarging means moving these arrays: about 31 accesses to models, 21 to textures, 4 to contours through gGfxUnkPtrs, 8 through gLevelData, the boot loop and the loaders (uvLevelAppend, uvMemLoadDS), plus dobj checks `modelId >= 0x190`. Large; only needed when new islands bring more models or textures than the free 37 / 27 / 37.

6. Photo missions (snap.c)

- **Correction of Mission_Scoring.md 1.1:** the curve input is the **distance between camera and subject** (func_8034B2B0 of subject − camera), not the animation time; the animation time (…GetInterval) is only stored in the photo. Points of a photo = PHTS points (+0x10) × centring factor (1.1 − distance of the subject from the picture's centre, at most 1; outside the middle half of the picture 0) × curve(distance); with PHTS mode 1 (+4) only within ±tolerance/2 (+0xC) of the angle (+8).
- Six main subjects with a curve each in COMM: ferry [5] 0x1A4, fountain [6] 0x1F8, missile [7] 0x24C, flame/oil plant [8] 0x2A0, shuttle [9] 0x2F4, whale [10] 0x348 (offsets from COMM+0x48); D_80350510 = {4, 3, 1, 2, 6, 5} is the search order, the nearest visible one is the subject.
- Types 7–10 (whale pod 5, planes 2, glider toys 4, boats 2; func_803386A4) are only recorded as further objects of a photo and never score. They are the natural candidates for curves 7–10 ("room for 10 photo curves").

6.1 How a photo scores (snapPhoto func_80338A14, func_80338614, checked in detail)

1. **Subject:** of the six main subjects that are active, in front of the camera (func_80338964) and projected within −50…370 × −50…290 pixels, the **nearest** becomes the photo's subject (unk43[0]); with none the photo shows "nothing" (11). A nearer other subject therefore takes the photo even if the mission wants another one.
2. **Value**, only if the mission has a PHTS entry of that type (for several entries of the type the last one counts):
   - the subject must lie in the middle half of the picture (x and y between 25 % and 75 %, in front of the camera), otherwise 0;
   - with PHTS mode 1 (+4) the view direction's heading must lie within angle (+8) ± tolerance/2 (+0xC) degrees, otherwise nothing is stored;
   - centring factor $f = \min(1, 1.1 - \sqrt{(dx^2 + dy^2)})$ with $dx, dy = 1 - (4 \times \text{position} - 1)$, i.e. full value within 2.5 % of the picture size around the centre, falling linearly to 0.1 at the edge of the middle half (on one axis) and to 0 or less towards its corners;
   - stored: unk38 = PHTS points (+0x10) × f, unk3C = curve(distance camera–subject in game units, × 0.7 m).
3. **Best photo per subject:** after every photo, of all photos of the same subject type only the one with the highest unk38 × unk3C is marked (unk6C); negative or zero values are never marked.
4. **Points** (flag 0x2000, func_80338614): the sum of round(unk38 × unk3C) of the marked photos (one per subject type), at most 6 photos per flight (PHOTO_COUNT_MAX).

- Retail split each subject's points like the Gyrocopter's landing: PHTS points = curve maximum = √points (HG A1 flame 7.746 → 60, HG B2 fountain and whale 5.477 → 30 + 30, HG P3 ferry, missile, shuttle 4.472 → 20 + 20 + 20). HG B2's fountain uses mode 1 (90° ± 130°). The three Birdman "photo missions" have no PHTS entry: their photos give 0 points (they only have the album button).
- Distances of the retail curves (game units): flame full 100–120, 0 beyond 500; fountain full 65–110; whale full 80–130; ferry full 180–270; missile full 50–80; shuttle full 230–280.

7. Radar (hud.c)

- **Waypoints** (rings, balloons, targets …, waypoints[70], hudAddWaypoint): each frame the radar draws only the **10 nearest** (the loop `for i < 10` picks the nearest not yet drawn).
- **Goals** (hopper goals, ball targets, used landing pads, landing strips): hud_8031A378 fills goals[10] **once at the start**, all of them, without a check; all are drawn every frame. More than 10 overwrite the waypoints behind them.

8. Display objects per island (measured, Gyrocopter missions)

Island (mission)	dobjs in flight	rings of the mission	rest (island, vehicle, effects)
Holiday (GC Beginner)	13	3	about 10
Crescent (GC A1)	39	15	about 24
Little States (GC B1)	52	22	about 30
Ever-Frost (GC P1)	66	24	about 42

One snapshot each; animated island objects and effects change the count. A mission on Ever-Frost Island therefore has room for roughly 55 display objects of its own.

9. Notes for the map editor (to go into its handover document)

- Space per island: precomputed figures of the four islands; when models are added, the editor computes the island's sum at once (files, unpacked) against the budget of 5.8 MB (minus vehicle, pilot, HUD, objects).
- Display list scan: a circle slightly larger than the view distance moves over the island; at each point the editor sums the display list commands of everything within the circle (terrain tiles, models, the mission's objects) plus a measured fixed part (HUD, sky, vehicle) and shows the worst places against 4200. Needs the parsing of models and terrain the map editor needs anyway; calibrate against emulator measurements (uvGfxEnd's DL_ELEMENTS).
- A warning when a mission's objects get near a sensible limit, suggesting the scan.
- The "raise limit" button per object type (slider, at most +10 at a time, cap and shared budget shown) lives where objects are added (map editor), unless it is built earlier.
- **Open user tests (user, 26 October 2026):** the user tested only the photos of Step 47 so far; the tests of Steps 44–46 (object memory, 256 display objects, raised object limits with many rings/balloons …, map view with many objects, the Missions tab warnings, import of raised limits, long flights with much graphics, radar goals, the stop without Expansion Pak) are done once the map editor can place many objects. Put them into the map editor's handover document as a test list.

10. Decisions of the user (26 October 2026)

- Object memory of a mission: 32 KB (moved to the high data area, always built in).
- Display objects: raised from 100 to 256 (revised the same day; the kernel's dobj arrays and the part-matrix pool 300 → 768 move to the high data area, about 66 KB).
- Raisable limits (upper caps): rings 30 → 64, balloons 20 → 40, hover pads 20 → 40, missile targets 40 → 64, thermals 16 → 32, local winds 6 → 16. Landing pads (14), landing strips (2), ball targets (5) and hopper goals (5) stay (retail missions use at most one of each; the 14 landing pads are those of the mission, which snap to the island's own pad places within 100 units, also at most 14).
- Raising happens in small steps: at most 10 at a time, the exact number chosen with a

slider; the dialog shows the type's cap and what the mission's shared budget still allows (display objects with the island's base load, radar waypoints 70, object memory). No space is reserved in advance: the game side gets lists exactly as large as the project's limits, and only when a limit is raised.

- Radar goals: the game module keeps all goals and enters the 10 nearest every frame (built in).
- Display list buffer: enlarged (cheap); the 4200 is only the size of a RAM buffer, not a hardware limit (more commands cost drawing time, i.e. slowdown, not a freeze).
- File tables: with the model editor, which comes after the map editor (the map editor uses existing models only).
- Photos: a button next to the photo points opens a window with the explanation of section 6.1; Birdman missions can be photo missions; the ten curve slots of a mission can be assigned freely to subjects (type and, for several objects of a type, the single object); default for slots 7–10: whale pod, planes, glider toys, boats.

10.1 Further findings and decisions (26 October 2026, second round)

- **Display objects** (dobjs) are all movable objects drawn: vehicle, island toys (boats, planes, ski lift …), effects and the mission's objects. The static island (terrain, buildings) is not counted.
- **Display list and drawing time:** everything shares the buffer and the frame time (terrain and island models, objects, sky, HUD). More commands cost frame rate. The editor keeps a **budget**: the map's peak need (from the scan) plus a measured fixed part is reserved first, the mission's objects get the rest ("the map has priority"). This is bookkeeping in the editor; the game itself just fills the buffer in drawing order.
- Scan: runs in the background after placing objects; when it shows plenty of room, the editor remembers how much can still be placed before the next scan is needed. When a model is imported (model editor), a one-time scan of the map warns about bad spots and colours the triangles by load (red = critical).
- **Photos per flight:** PHOTO_COUNT_MAX 6 is shared by the flight's photos (D_80373390) and the album (D_80373060, saved). The flight can take 10 (12 code references and the constants of snap.c); the album stays 6 because the save file has no room for 4 more photos (4 × 174 bits) with 128 missions.
- **Example photos:** the test screen's photo button (only for photo missions) shows example photos **per island**, not per mission: snapInit places a camera at fixed offsets from the subject (Crescent: whale, fountain; Little States: missile, ferry, shuttle; Ever-Frost: flame; Holiday: none) and the album viewer renders them live (func_8033E3A8(3), func_8033DDD8). They are not scored. For free photo subjects the module provides the mission's own examples; the editor computes each one from the subject's curve (distance of the curve's maximum), the angle setting and a centred view, and lets the user adjust distance, height and direction.

11. Built in Step 44: high data area, display list, object memory, radar goals

- **High data area 0x80780000–0x80800000** (`HighDataArea`, every hack): uvLevelInit sets the heap's end to 0x80780000 instead of osMemSize + 0x80000000 (ROM 0x2BEFC `lui t8, 0x8078`).
  - 0x80780000: the two display lists, 8192 commands each (64 KiB per frame).
  - 0x807A0000: the object memory of a mission, 32 KiB (`ObjectMemoryLimit` 0x7FF8 is mem_get's check).
  - 0x807A8000: free for the next tables.
- **Display list:** uvGfxInit, uvGfxEnd (count, check against 8192, the message, the task's data pointer) and the head of the next frame use the new base; the chains computing frame × 4200 × 8 became one shift by 16. Developers' warning threshold 7800.
- **Object memory:** mem_get, mem_init, mem_remaining and the start pointer (ROM 0x99CA4…0x99D2C, 0xD6D70). The build refuses a mission whose objects need more (`MissionObjects.Problem`; the game would crash).
- **Radar goals:** hud_8031A378 writes into `module_radar_goals` (room for 32); the six calls of hudDrawRadar and the module's Birdman HUD go through `module_hud_radar` (objects.c), which copies the 10 goals nearest to the player into the radar.
- **Checked:** emulator (mupen64plus, Gyrocopter on all four islands): heap end 0x80780000, the display list's head alternating between 0x80780000 and 0x80790000, the object memory's pointer at 0x807A0000 + 488 / 2072 / 2992 / 3256, the runway as the only goal in the module's list and in the radar, radar and HUD drawn as before; with 4 MB the module's "No Expansion Pak" stop as before. Not checked: a mission with more than 10 goals (none exists yet).

12. Built in Step 45: 256 display objects, 1024 matrices per frame

- **Layout of the high data area** behind the object memory: 0x807A8000 display objects (256 × 0x40), 0x807AC000 part matrices (768 × 0x40), 0x807B8000 parts in use (768), 0x807B8300 draw list (256 × u16), 0x807B8500 drawn (256 × s32), 0x807B9000 matrix stack (2 × 1024 × 0x40); free from 0x807D9000.
- **References:** 60 lui/addiu pairs and 5 indexed accesses ("lui at, hi; addu at, at, index; lhu/sh/sb/sw x, lo(at)", e.g. 0x80200670 reading an object's model id) in the kernel. A first build without the indexed ones crashed (model 0 read from the old, empty table); a scan of the whole code (also following `addu`) and of all data words found no further references. The loops over a table compare with its end, which is the next table's start: those 6 were told apart by reading the code (uvDobjInit, _uvDobjsDraw, uvGfxEnd's statistics).
- **Counts:** objId checks 100 / 101 → 256 (uvDobjModel, uvDobjProps, uvDobjPosm, uvDobjGetPosm, uvDobjState …), the allocation loop, the two loops of code_8170.c (collision with display objects), the part search 300 → 768.
- **Matrix stack:** the checks against 350 and the frame offsets (frame × 350 × 64 as a chain → one shift by 16).
- **Developers' warnings** now start at 960 matrices, 700 part matrices, 240 display objects.

- **Checked:** emulator: Gyrocopter on all four islands (the same number of display objects as before, 13 / 39 / 52 / 66, now in the new table; 29–155 part matrices in use), Gyrocopter Pilot flight for a minute, Rocket Belt Beginner flight with the wall hit ("2 pts. deducted") and the balloon, the same as the build of Step 44. Not checked: a mission with more than 100 display objects (needs the map editor).

13. Built in Step 46: raised object limits, map view, drawn objects, mission checks

- **Project setting** `ObjectLimits` (type name → limit, e.g. "Ring": 40; `ObjectLimits` in Core): rings 30 → 64, balloons 20 → 40, hover pads 20 → 40, missile targets 40 → 64, thermals 16 → **20** (the radar copies all thermals into HUDRadar.therms[20] without a check), local winds 6 → 16. `ObjectLimits.Raise` allows at most +10 at a time (`NextMaximum`); the user interface (slider) comes with the map editor.
- **Game side** (only for raised types): the list moves to the high data area (from 0x807DAE00, exactly the project's size, in the order of `ObjectLimits.Lists`); all references (80 in the app code: lui/addiu and indexed accesses; the init loops of rings, hover pads, targets and thermals run to the list's end) and the loaders' checks ("too many … defined in level", ringsLoad slti 31 …; ballsInit's count and the split check of balls_802CB9B4) follow. The words behind a list are other variables (gRingChildIndexes behind gRings, gThermReady behind gThermals, a float behind gBalls) and keep their addresses. The module's conditions read a raised ring or balloon list from the data area header (+0x28 ring list and limit, +0x30 balloon list). The import reads the limits back from the loaders' checks.
- **Always built in:** the map view's object list (sMapObjects, 60 entries without a check, more overwrote sMapMessages) has 256 entries at 0x807D9000; the list of display objects drawn per frame (_uvSortAdd, D_802629C8, 60: more were not drawn, "_uvSortAdd: dobj list full") has 256 at 0x807D9200.
- **Build checks:** a mission with more objects of a type than its list allows is refused (the game would load none of them), as one that needs more object memory.
- **Missions tab warnings** (`MissionBudget`): the mission's display objects (estimate: one per ring, balloon, hover pad, target, ball target, hopper goal, landing pad, one for Meca Hawks) plus the island's own (`IslandBudget`: 10 / 24 / 30 / 42) above 256; the mission's objects on the radar above 70.
- **Island values** (`IslandBudget`): files unpacked and packed, heap in flight, display objects of the island, and the heap of a hack (5.3 MB up to the high data area) – for the model editor's space display.
- **Checked:** emulator: Gyrocopter Class A with 40 and with 64 rings (limits 40 / 64): all rings loaded into the new list (sRingsCount 40 / 64), 64 / 88 display objects, object memory 5368 bytes (more than the game's 5000); the map view of the 64-ring mission without "dobj list full" (with only Step 45 it printed it 1117 times). Not flown through the extra rings by hand.

14. Built in Step 47: ten photos per flight, photo grid with pages, "Please wait..."

Division of the photo work (decided by Claude, the user left the steps open): Step 47 the flight

side (ten photos, grid with pages, "Please wait..."); next the photo subjects (the ten curve slots, free subjects with single objects, Birdman photo missions, the photo points in the Missions tab with the explanation window); then the example photos per mission (the game computes them per mission; the editor's tool to pick a full-points example with a live picture probably comes with the map editor, which brings the island rendering it needs).

- **Flight photos** (`FlightPhotos`, every hack): D_80373390[6] moves to **0x807F0000** (`HighDataArea.FlightPhotoTable`, 10 × 0x70, behind the room the raised object lists can need at most; 4 KiB kept for the photo tables of the next steps). References: snapInit (base and loop end, it clears ten), func_80338614 (points), snapPhoto (the new photo, the best-photo loop, the indexed store of unk6C "lui at; addu at, at, index; sw t5, 0x33FC(at)"), func_8033E3A8 (grid and single view). Counts 6 → 10: snapPhoto, the Hang Glider's shutter (two checks), the Birdman's shutter, snapGetPhotoCount (HUD 10 … 0). The album stays at six (D_80373060; the "room in the album" checks of func_8033DFD0 are unchanged). func_80337D50 is left alone: the code fix photo-album-v1 already points it to the album.
- **Photo grid** (func_8033D3EC → `module_photo_grid`, photos.c, built with the FPU like weather.c): pages of six photos. Right from the right column turns to the next page (left column, same row), left from the left column to the previous one (right column), cyclic; yellow arrows at both ends of the title bar when there is more than one page. The page and the chosen photo stay when the single view returns. With six photos or fewer the grid behaves as the game's (the same moves, including the odd diagonal wrap).
- **"Please wait..."** (module header +0x180, the game's text codes from `ProjectBuilder.BusyText`, English like all game texts): in the grid in the title bar until the six photos are drawn (then the bar is drawn again and "Check which photo ?" appears); in the single views (example photos func_8033DDD8, one photo func_8033DFD0) dark in the middle of the white photo area through `module_photo_busy` instead of func_8033A610; the picture replaces it. When the photo's test has to be loaded (func_8033A128 → gameUpdateStateTestSetup), that function first clears the photo area twice (black) and then loads the island for seconds: the user saw the text only for a moment (ares). Rework: both calls of func_8033A128 in the single views go through `module_photo_setup`, and the second uvGfxEnd of gameUpdateStateTestSetup (ROM 0x73ABC) through `module_setup_frame`, which draws the text again in white on the cleared area while a photo is set up; it stays until the picture is drawn.
- Note: the film strip's bar with "Check which photo ?" (y 191–213 in the ortho coordinates) is at the **top** of the screen.
- **Module size:** 44960 bytes, up to 0x8038E980; 6 KiB are left in front of the user files (0x803901E0). The photo subjects and example photos will need more: a second code part in the high data area is planned for them.
- **Checked:** emulator (mupen64plus): Hang Glider Beginner, 11 presses of Z: HUD 10 → 0, ten photos (page 1 six, page 2 four and two grey places), "Please wait..." in the title bar while drawing; right over the edge turns to page 2, left back to page 1 at the right column; a photo of page 2 opens in the single view, B returns to page 2; "Yes" puts it into the

album; the album via Options → Check album (four photos, one page, no arrows, "Please wait..."), a single album photo and back; the sample photo of HG Class A test 1 (Flame) with "Please wait..." in the photo area while loading, after the rework also on the black area during the whole load (frame by frame). User (ares): photos tested; the button for the photo explanation is not there yet (it comes with the photo subjects step). Not checked: the penalty sound of the eleventh press (no sound in the test runs).